

Sistemas Operativos

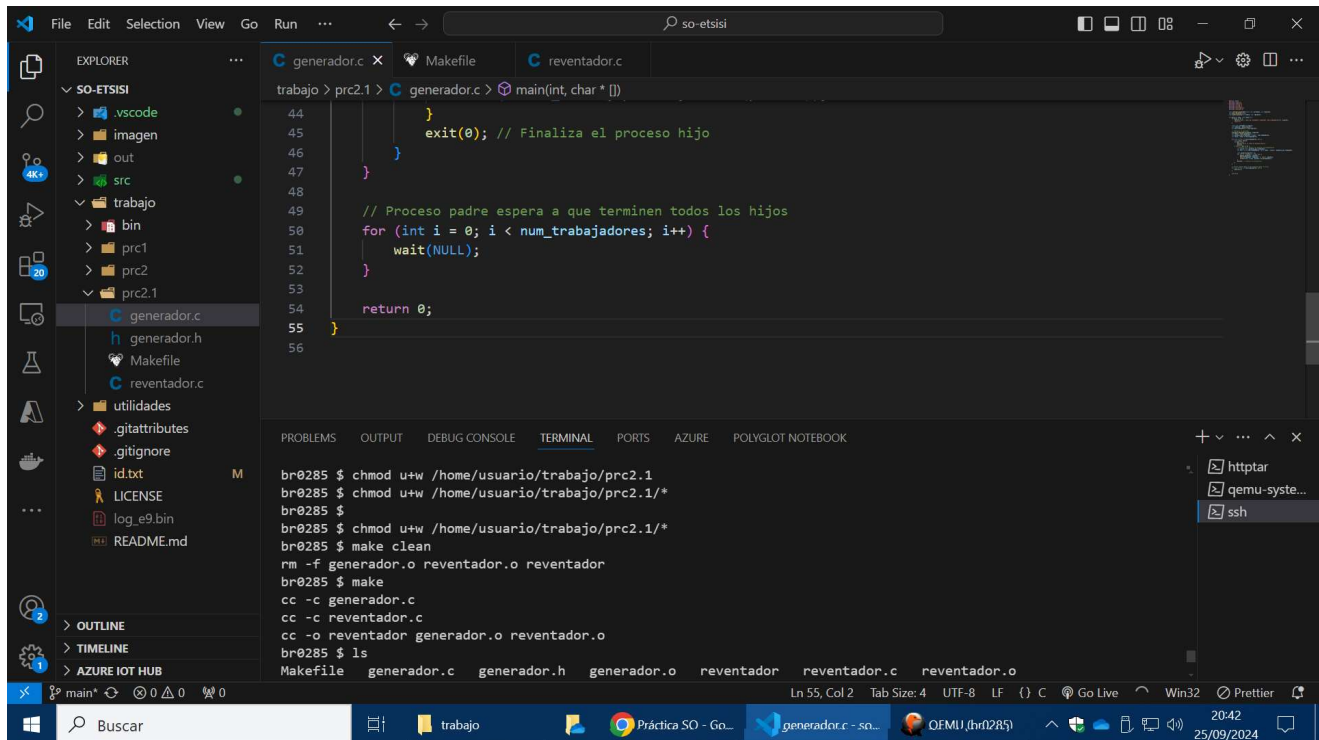
Práctica 2.1

Fecha: 29/09/2024

Alumno: Yzan Martinez Manzano

Correo: yzan.martinez.manzano@alumnos.upm.es

1.-Llamadas a sistema



The screenshot shows a Visual Studio Code editor window with a project named 'so-etsisi'. The Explorer sidebar on the left shows the project structure, including a 'trabajo' directory with subdirectories 'prc1', 'prc2', and 'prc2.1'. The 'prc2.1' directory contains files 'generator.c', 'generator.h', 'Makefile', and 'reventador.c'. The main editor window displays the 'generator.c' file, which contains the following C code:

```
44     }
45     exit(0); // Finaliza el proceso hijo
46 }
47
48
49 // Proceso padre espera a que terminen todos los hijos
50 for (int i = 0; i < num_trabajadores; i++) {
51     wait(NULL);
52 }
53
54 return 0;
55 }
56
```

Below the editor, the TERMINAL panel shows the execution of the program in a shell environment:

```
br0285 $ chmod u+w /home/usuario/trabajo/prc2.1
br0285 $ chmod u+w /home/usuario/trabajo/prc2.1/*
br0285 $
br0285 $ chmod u+w /home/usuario/trabajo/prc2.1/*
br0285 $ make clean
rm -f generator.o reventador.o reventador
br0285 $ make
cc -c generator.c
cc -c reventador.c
cc -o reventador generator.o reventador.o
br0285 $ ls
Makefile generator.c generator.h generator.o reventador reventador.c reventador.o
```

The status bar at the bottom indicates the current file is 'main*' and the editor is using the 'UTF-8' encoding and 'LF' line endings. The system tray at the bottom right shows the date and time as '20:42 25/09/2024'.

This screenshot shows the Visual Studio Code editor with a project named 'so-etsisi'. The Explorer sidebar on the left shows a directory structure with folders like 'imagen', 'out', 'src', 'trabajo', and sub-directories 'prc1', 'prc2', and 'prc2.1'. The 'prc2.1' folder is expanded, showing files 'generator.c', 'generator.h', 'Makefile', and 'reventador.c'. The main editor window displays the code in 'generator.c', which includes a worker process function that calls 'exit(0)' and a main function that waits for all child processes to finish using 'wait(NULL)'. The bottom panel shows the 'TERMINAL' tab with a shell prompt 'br0285 \$' and the command 'tail \$HOME/original.txt' being executed, showing a list of characters. The status bar at the bottom indicates the current file is 'main.c' at line 55, column 2.

```
44     }
45     exit(0); // Finaliza el proceso hijo
46 }
47
48 // Proceso padre espera a que terminen todos los hijos
49 for (int i = 0; i < num_trabajadores; i++) {
50     wait(NULL);
51 }
52
53 return 0;
54 }
55
56
```

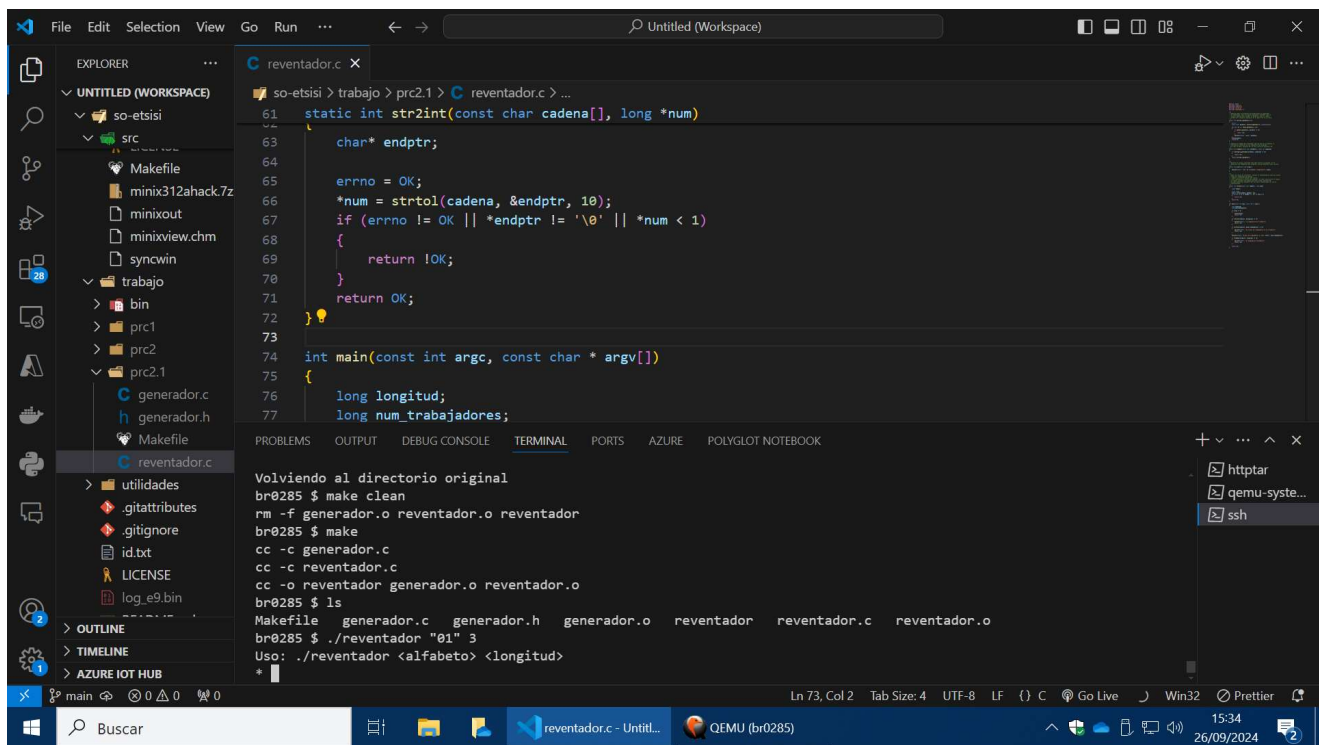
br0285 \$ tail \$HOME/original.txt
zzq
zzr
zss
zzt
zzu
zzv
zzw
zzx
zzy
zzz
br0285 \$

2.-Argumentos del programa

This screenshot shows the Visual Studio Code editor with a project named 'so-etsisi'. The Explorer sidebar on the left shows a directory structure with folders like 'so-etsisi', 'src', 'trabajo', and sub-directories 'prc1', 'prc2', and 'prc2.1'. The 'prc2.1' folder is expanded, showing files 'generator.c', 'generator.h', 'Makefile', and 'reventador.c'. The main editor window displays the code in 'reventador.c', which includes a function 'str2int' for parsing integers from a string and a main function that uses this function to parse command-line arguments. The bottom panel shows the 'TERMINAL' tab with a shell prompt 'br0285 \$' and a series of commands: 'make clean', 'make', 'ls', and 'ls', which are used to build and run the program. The status bar at the bottom indicates the current file is 'reventador.c' at line 73, column 2.

```
61 static int str2int(const char cadena[], long *num)
62 {
63     char* endptr;
64
65     errno = OK;
66     *num = strtol(cadena, &endptr, 10);
67     if (errno != OK || *endptr != '\0' || *num < 1)
68     {
69         return 10K;
70     }
71     return OK;
72 }
73
74 int main(const int argc, const char * argv[])
75 {
76     long longitud;
77     long num_trabajadores;
```

br0285 \$ make clean
rm -f generator.o reventador.o reventador
br0285 \$ make
cc -c generator.c
cc -c reventador.c
cc -o reventador generator.o reventador.o
br0285 \$ ls
Makefile generator.c generator.h generator.o reventador reventador.c reventador.o
br0285 \$



```
#define _MINIX
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include "generador.h"
/**
 * Muestra todas las palabras que puede genera el generador.
 * Es importante, para los pasos siguientes, entender cómo se
 * reserva memoria para recoger la palabra y cómo se libera,
 * porque será necesario aplicarlo en el paso 5 de la práctica.
 */
static int escribir_palabras(void)
{
    int i;
    char* const palabra = calloc(longitud()+2, sizeof(char)); //Reserva memoria dinámica
    para almacenar la palabra

    for (i = 0; i < total_palabras(); i++)
    {
        if (genera_palabra(i, palabra) != OK)
        {
            return !OK;
        }
        fprintf(stdout, "%s\n", palabra);
    }
    free(palabra); //Libera la memoria reservada
    return OK;
}
/* Realiza el trabajo del reventador una vez que se ha obtenido la
 * información necesaria de los argumentos del programa.
 * Si todo va bien, devuelve OK. En caso contrario devuelve !OK.
```

```

*/
static int trabajar(const char alfabeto[], const int longitud)
{
    if (configura_generador(alfabeto, longitud) != OK)
    {
        return !OK; //Retorna !OK si no funciona el generador
    }
    return escribir_palabras(); //Genera y enseña las palabras
}

/**
 * Muestra un mensaje indicando cómo debe usarse el programa. Si se
 * modifican los argumentos del programa, hay que modificar esta función.
 */
static void uso(const char prog[])
{
    fprintf(stderr, "Uso: %s <alfabeto> <longitud>\n", prog);
}

/**
 * Dada una cadena de caracteres, obtiene su representación numérica entera.
 * Tiene los siguientes parámetros:
 * cadena: la versión textual del número.
 * num: referencia al entero que albergará el valor numérico de la cadena.
 * Si cadena contiene una cadena numérica válida, devuelve OK.
 * En caso contrario, devuelve !OK y el valor referenciado por num es
 * indeterminado.
 */
static int str2int(const char cadena[], long *num) { //Convierte una cadena a un número
entero.
    char* endptr;
    errno = OK;
    *num = strtol(cadena, &endptr, 10);
    if (errno != OK || *endptr != '\0' || *num < 1)
    {
        return !OK;
    }
    return OK;
}

int main(const int argc, const char * argv[])
{
    long longitud;
    long num_trabajadores;

    if (argc != 4)
    {
        uso(argv[0]);
        return !OK;
    }

    if (str2int(argv[2], &longitud) != OK)
    {
        fprintf(stderr, "La longitud no es válida\n");
    }
}

```

```
        return !OK;
    }

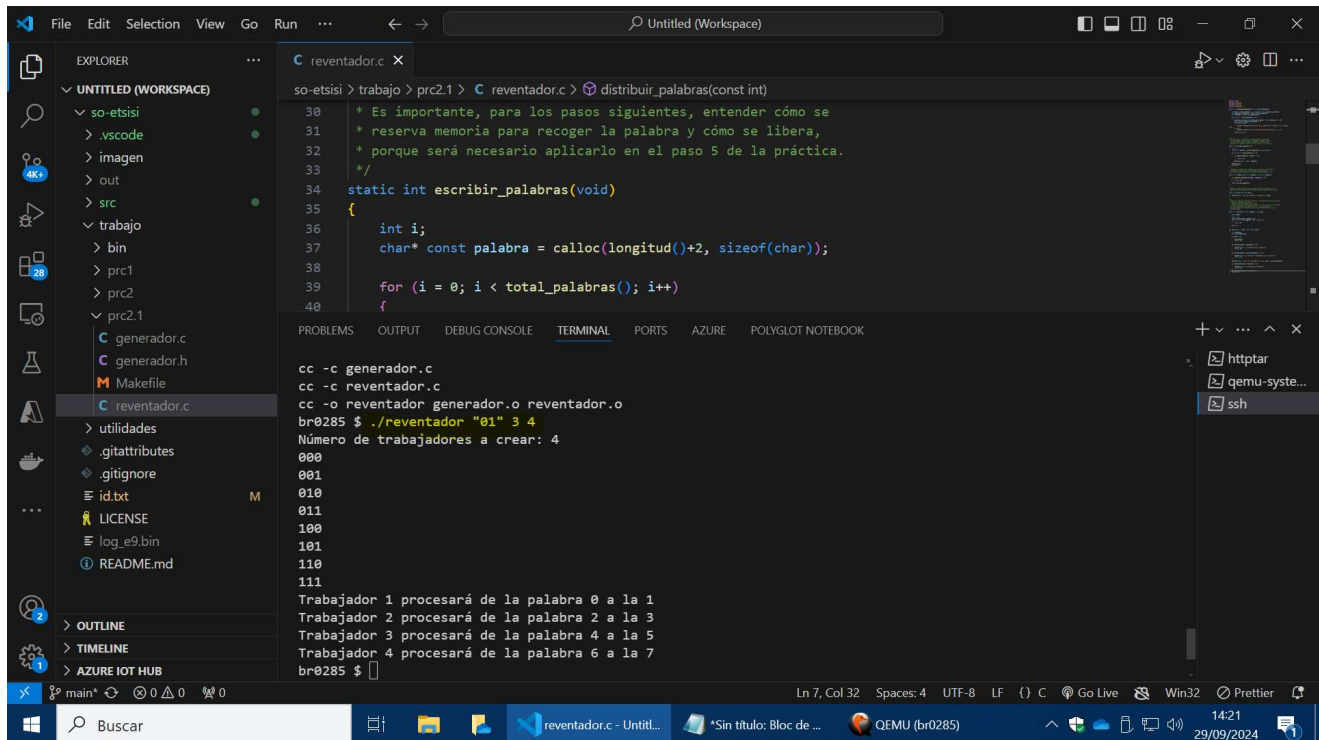
    if (str2int(argv[3], &num_trabajadores) != OK)
    {
        fprintf(stderr, "El número de trabajadores no es válido\n");
        return !OK;
    }

    fprintf(stdout, "Número de trabajadores a crear: %ld\n", num_trabajadores);

    if (trabajar(argv[1], longitud) != OK)
    {
        fprintf(stderr, "El programa ha fallado\n");
        return !OK;
    }

    return OK;
}
```

3.-Número de palabras



```
so-etsisi > trabajo > prc2.1 > C reventador.c > distribuir_palabras(const int)
```

```
30  * Es importante, para los pasos siguientes, entender cómo se
31  * reserva memoria para recoger la palabra y cómo se libera,
32  * porque será necesario aplicarlo en el paso 5 de la práctica.
33  */
34  static int escribir_palabras(void)
35  {
36      int i;
37      char* const palabra = calloc(longitud()+2, sizeof(char));
38
39      for (i = 0; i < total_palabras(); i++)
40      {
```

```
cc -c generator.c
cc -c reventador.c
cc -o reventador.generator.o reventador.o
br0285 $ ./reventador "01" 3 4
Número de trabajadores a crear: 4
000
001
010
011
100
101
110
111
Trabajador 1 procesará de la palabra 0 a la 1
Trabajador 2 procesará de la palabra 2 a la 3
Trabajador 3 procesará de la palabra 4 a la 5
Trabajador 4 procesará de la palabra 6 a la 7
br0285 $
```

```
#define _MINIX
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include "generator.h"

static void distribuir_p_trabajadores(const int num_trabajadores)
{
    int total = total_palabras();
    int i, fin;
    int inicio = 0;
    int palabras_a_procesar;
    int palabras_por_trabajador = total / num_trabajadores;
    int restantes = total % num_trabajadores;

    for(i = 0; i < num_trabajadores; i++)
    {
        palabras_a_procesar = palabras_por_trabajador + (i < restantes ? 1 : 0);
        fin = inicio + palabras_a_procesar - 1;
        if(inicio < total)
        {
            printf("Trabajador %d procesará de la palabra %d a la %d\n", i + 1,
inicio,fin);
        } else {
            printf("Trabajador %d no tiene palabras para procesar\n", i + 1);
        }
        inicio = fin + 1;
    }
}
```

```

}

/**
 * Muestra todas las palabras que puede genera el generador.
 * Es importante, para los pasos siguientes, entender cómo se
 * reserva memoria para recoger la palabra y cómo se libera,
 * porque será necesario aplicarlo en el paso 5 de la práctica.
 */
static int escribir_palabras(void)
{
    int i;
    char* const palabra = calloc(longitud()+2, sizeof(char));

    for (i = 0; i < total_palabras(); i++)
    {
        if (genera_palabra(i, palabra) != OK)
        {
            return !OK;
        }
        fprintf(stdout, "%s\n", palabra);
    }
    free(palabra);
    return OK;
}

/* Realiza el trabajo del reventador una vez que se ha obtenido la
 * información necesaria de los argumentos del programa.
 * Si todo va bien, devuelve OK. En caso contrario devuelve !OK.
 */
static int trabajar(const char alfabeto[], const int longitud)
{
    if (configura_generador(alfabeto, longitud) != OK)
    {
        return !OK;
    }
    return escribir_palabras();
}

/**
 * Muestra un mensaje indicando cómo debe usarse el programa. Si se
 * modifican los argumentos del programa, hay que modificar esta función.
 */
static void uso(const char prog[])
{
    fprintf(stderr, "Uso: %s <alfabeto> <longitud>\n", prog);
}

/**
 * Dada una cadena de caracteres, obtiene su representación numérica entera.
 * Tiene los siguientes parámetros:
 * cadena: la versión textual del número.
 * num: referencia al entero que albergará el valor numérico de la cadena.
 * Si cadena contiene una cadena numérica válida, devuelve OK.
 * En caso contrario, devuelve !OK y el valor referenciado por num es

```



```

* indeterminado.
*/
static int str2int(const char cadena[], long *num)
{
    char* endptr;

    errno = 0;
    *num = strtol(cadena, &endptr, 10);
    if (errno != 0 || *endptr != '\0' || *num < 1)
    {
        return !OK;
    }
    return OK;
}

int main(const int argc, const char * argv[])
{
    long longitud;
    long num_trabajadores;

    if (argc != 4)
    {
        uso(argv[0]);
        return !OK;
    }

    if (str2int(argv[2], &longitud) != OK)
    {
        fprintf(stderr, "La longitud no es válida\n");
        return !OK;
    }

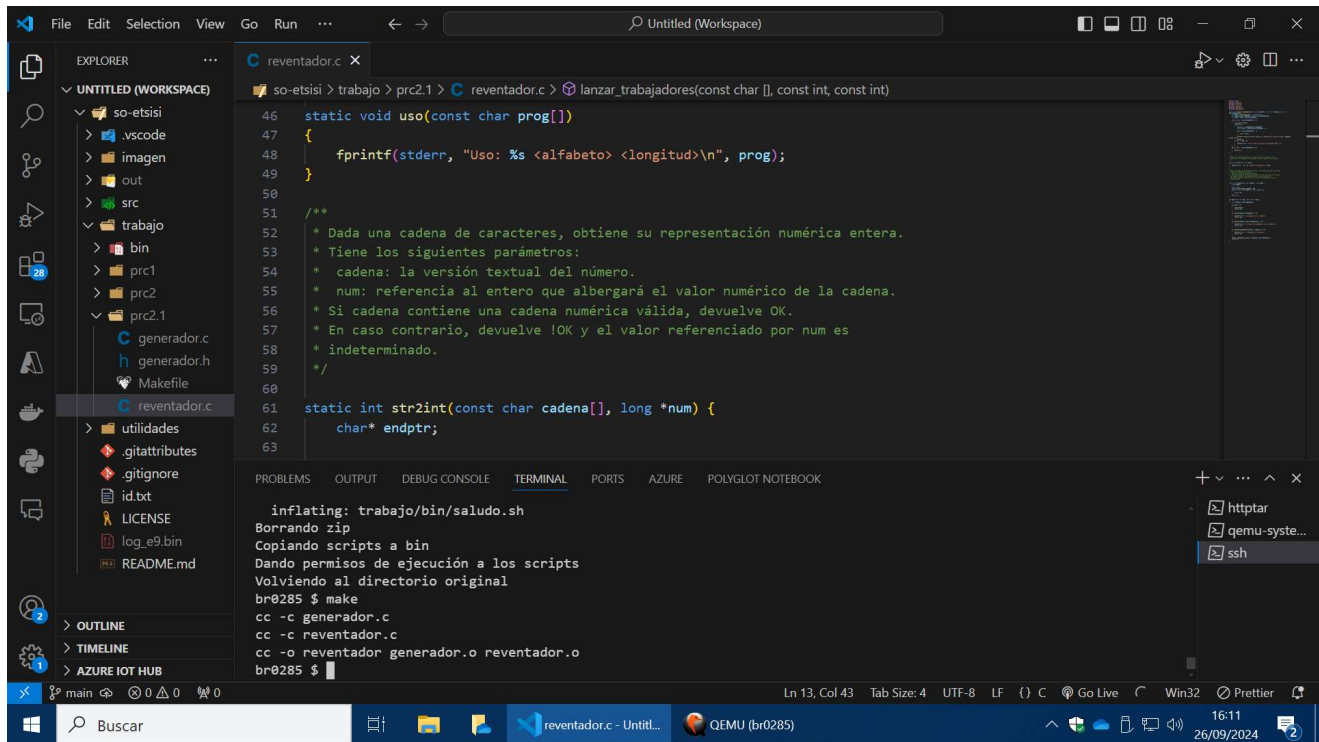
    if (str2int(argv[3], &num_trabajadores) != OK)
    {
        fprintf(stderr, "El número de trabajadores no es válido\n");
        return !OK;
    }

    fprintf(stdout, "Número de trabajadores a crear: %ld\n", num_trabajadores);

    if (trabajar(argv[1], longitud) != OK)
    {
        fprintf(stderr, "El programa ha fallado\n");
        return !OK;
    }
    distribuir_p_trabajadores(num_trabajadores);
    return OK;
}

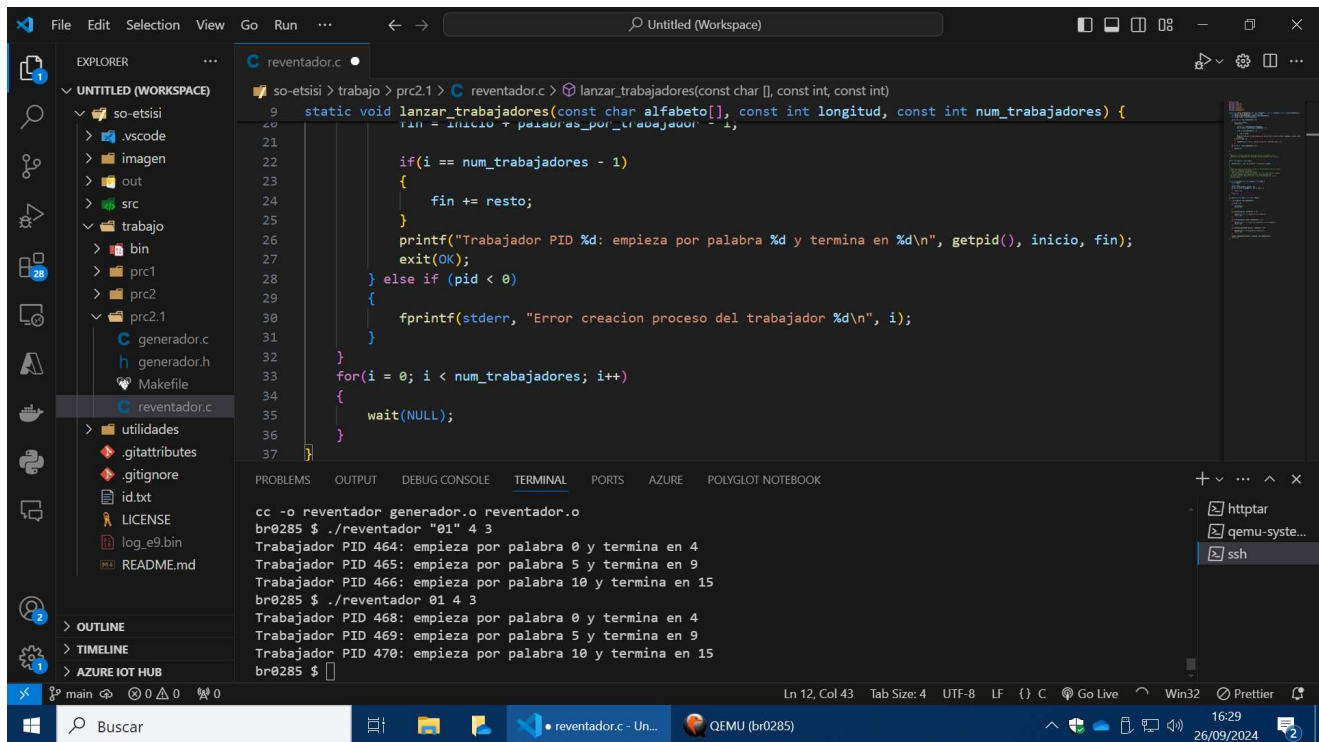
```

4.-Ejecución de los trabajadores



```
so-etisi > trabajo > prc2.1 > C reventador.c > lanzar_trabajadores(const char [], const int, const int)
46 static void uso(const char prog[])
47 {
48     fprintf(stderr, "Uso: %s <alfabeto> <longitud>\n", prog);
49 }
50
51 /**
52  * Dada una cadena de caracteres, obtiene su representación numérica entera.
53  * Tiene los siguientes parámetros:
54  * cadena: la versión textual del número.
55  * num: referencia al entero que albergará el valor numérico de la cadena.
56  * Si cadena contiene una cadena numérica válida, devuelve OK.
57  * En caso contrario, devuelve !OK y el valor referenciado por num es
58  * indeterminado.
59  */
60
61 static int str2int(const char cadena[], long *num) {
62     char* endptr;
63
```

inflating: trabajo/bin/saludo.sh
Borrando zip
Copiando scripts a bin
Dando permisos de ejecución a los scripts
Volviendo al directorio original
br0285 \$ make
cc -c generator.c
cc -c reventador.c
cc -o reventador generator.o reventador.o
br0285 \$



```
9 static void lanzar_trabajadores(const char alfabeto[], const int longitud, const int num_trabajadores) {
20     fin = inicio + palabras_por_trabajador - 1;
21
22     if(i == num_trabajadores - 1)
23     {
24         fin += resto;
25     }
26     printf("Trabajador PID %d: empieza por palabra %d y termina en %d\n", getpid(), inicio, fin);
27     exit(OK);
28 } else if (pid < 0)
29 {
30     fprintf(stderr, "Error creacion proceso del trabajador %d\n", i);
31 }
32 }
33 for(i = 0; i < num_trabajadores; i++)
34 {
35     wait(NULL);
36 }
37 }
```

cc -o reventador generator.o reventador.o
br0285 \$./reventador "01" 4 3
Trabajador PID 464: empieza por palabra 0 y termina en 4
Trabajador PID 465: empieza por palabra 5 y termina en 9
Trabajador PID 466: empieza por palabra 10 y termina en 15
br0285 \$./reventador 01 4 3
Trabajador PID 468: empieza por palabra 0 y termina en 4
Trabajador PID 469: empieza por palabra 5 y termina en 9
Trabajador PID 470: empieza por palabra 10 y termina en 15
br0285 \$

```
#define _MINIX
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <unistd.h>
#include <sys/wait.h>
#include "generator.h"
```

```

static void lanzar_trabajadores(const char alfabeto[], const int longitud, const int
num_trabajadores) {
    int total = total_palabras();
    int i, inicio, fin;
    int palabras_por_trabajador = total / num_trabajadores;
    int resto = total % num_trabajadores;

    for(i = 0; i < num_trabajadores; i++)
    {
        pid_t pid = fork();
        if(pid == 0)
        {
            inicio = i * palabras_por_trabajador;
            fin = inicio + palabras_por_trabajador - 1;

            if(i == num_trabajadores - 1)
            {
                fin += resto;
            }
            printf("Trabajador PID %d: empieza por palabra %d y termina en %d\n",
getpid(), inicio, fin);
            exit(OK);
        } else if (pid < 0)
        {
            fprintf(stderr, "Error creacion proceso del trabajador %d\n", i);
        }
    }
    for(i = 0; i < num_trabajadores; i++)
    {
        wait(NULL);
    }
}

/**
 * Muestra un mensaje indicando cómo debe usarse el programa. Si se
 * modifican los argumentos del programa, hay que modificar esta función.
 */

static void uso(const char prog[])
{
    fprintf(stderr, "Uso: %s <alfabeto> <longitud>\n", prog);
}

/**
 * Dada una cadena de caracteres, obtiene su representación numérica entera.
 * Tiene los siguientes parámetros:
 *  cadena: la versión textual del número.
 *  num: referencia al entero que albergará el valor numérico de la cadena.
 * Si cadena contiene una cadena numérica válida, devuelve OK.
 * En caso contrario, devuelve !OK y el valor referenciado por num es
 * indeterminado.
 */

static int str2int(const char cadena[], long *num) {

```

```

char* endptr;

errno = OK;
*num = strtol(cadena, &endptr, 10);
if (errno != OK || *endptr != '\0' || *num < 1)
{
    return !OK;
}
return OK;
}

int main(const int argc, const char * argv[])
{
    long longitud, num_trabajadores;

    if (argc != 4)
    {
        uso(argv[0]);
        return !OK;
    }

    if (str2int(argv[2], &longitud) != OK)
    {
        fprintf(stderr, "La longitud no es válida\n");
        return !OK;
    }

    if (str2int(argv[3], &num_trabajadores) != OK)
    {
        fprintf(stderr, "El número de trabajadores no es válido\n");
        return !OK;
    }

    if (configura_generador(argv[1], longitud) != OK)
    {
        fprintf(stderr, "El programa ha fallado\n");
        return !OK;
    }

    lanzar_trabajadores(argv[1], longitud, num_trabajadores);
    return OK;
}

```

5.-Programa final

File Edit Selection View Go Run ...

EXPLORER

UNTITLED (WORKSPACE)

so-etsisi

.vscode

imagen

out

src

trabajo

bin

prc1

prc2

prc2.1

generador.c

generador.h

Makefile

reventador.c

utilidades

.gitattributes

.gitignore

id.txt

LICENSE

log_e9.bin

README.md

OUTLINE

TIMELINE

AZURE IOT HUB

main 0 0 0 0

reventador.c

```
1 static void lanzar_trabajadores(const char alfabeto[], const int longitud, const int num_trabajadores) {
2
3     printf("Trabajador PID %d: empieza por palabra %d y termina en %d\n", getpid(), inicio, fin);
4     exit(0);
5 } else if (pid < 0)
6 {
7     fprintf(stderr, "Error creacion proceso del trabajador %d\n", i);
8 }
9
10 for(i = 0; i < num_trabajadores; i++)
11 {
12     wait(NULL);
13 }
14
15 /**
16  * Muestra un mensaje indicando cómo debe usarse el programa. Si se
17  * modifican los argumentos del programa, hay que modificar esta función.
18  */
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE POLYGLOT NOTEBOOK

```
cc -c generador.c
cc -c reventador.c
cc -o reventador generador.o reventador.o
br0285 $ ./reventador abcdefghijklmnopqrstuvwxyz 3 4
Trabajador PID 566: empieza por palabra 0 y termina en 4393
Trabajador PID 567: empieza por palabra 4394 y termina en 8787
Trabajador PID 568: empieza por palabra 8788 y termina en 13181
Trabajador PID 569: empieza por palabra 13182 y termina en 17575
br0285 $ ./reventador abcdefghijklmnopqrstuvwxyz 3 4 > $HOME/original.txt
br0285 $
```

Ln, Col 11 Tab Size: 4 UTF-8 LF {} C Go Live Win32 Prettier

16:49 26/09/2024

```
#define _MINIX
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <unistd.h>
#include <sys/wait.h>
#include "generador.h"
```

```

static void lanzar_trabajadores(const char alfabeto[], const int longitud, const int
num_trabajadores) {
    int total = total_palabras();
    int i, inicio, fin;
    int palabras_por_trabajador = total / num_trabajadores;
    int resto = total % num_trabajadores;

    for(i = 0; i < num_trabajadores; i++)
    {
        pid_t pid = fork();
        if(pid == 0)
        {
            inicio = i * palabras_por_trabajador;
            fin = inicio + palabras_por_trabajador - 1;

            if(i == num_trabajadores - 1)
            {
                fin += resto;
            }
            //Modifico el como salen las palabras a los trabajadores con printf's
            for (int j = inicio; j <= fin; j++) {
                char palabra[longitud + 1];
                obtener_palabra(j, palabra);
                printf("%s\n", palabra);
            }

            exit(OK);
        } else if (pid < 0)
        {
            fprintf(stderr, "Error creacion proceso del trabajador %d\n", i);
        }
    }
    for(i = 0; i < num_trabajadores; i++)
    {
        wait(NULL);
    }
}

/**
 * Muestra un mensaje indicando cómo debe usarse el programa. Si se
 * modifican los argumentos del programa, hay que modificar esta función.
 */

static void uso(const char prog[])
{
    fprintf(stderr, "Uso: %s <alfabeto> <longitud> <num_trabajadores>\n", prog);
}

/**
 * Dada una cadena de caracteres, obtiene su representación numérica entera.
 * Tiene los siguientes parámetros:
 *   cadena: la versión textual del número.
 *   num: referencia al entero que albergará el valor numérico de la cadena.
 * Si cadena contiene una cadena numérica válida, devuelve OK.

```

```

* En caso contrario, devuelve !OK y el valor referenciado por num es
* indeterminado.
*/

static int str2int(const char cadena[], long *num) {
    char* endptr;

    errno = OK;
    *num = strtol(cadena, &endptr, 10);
    if (errno != OK || *endptr != '\0' || *num < 1)
    {
        return !OK;
    }
    return OK;
}

int main(const int argc, const char * argv[])
{
    long longitud, num_trabajadores;

    if (argc != 4)
    {
        uso(argv[0]);
        return !OK;
    }

    if (str2int(argv[2], &longitud) != OK)
    {
        fprintf(stderr, "La longitud no es válida\n");
        return !OK;
    }

    if (str2int(argv[3], &num_trabajadores) != OK)
    {
        fprintf(stderr, "El número de trabajadores no es válido\n");
        return !OK;
    }

    if (configura_generador(argv[1], longitud) != OK)
    {
        fprintf(stderr, "El programa ha fallado\n");
        return !OK;
    }

    lanzar_trabajadores(argv[1], longitud, num_trabajadores);
    return OK;
}

```